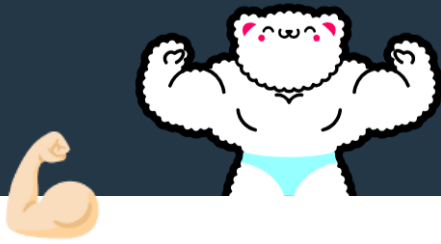


Smartgym

Analisis y diseño

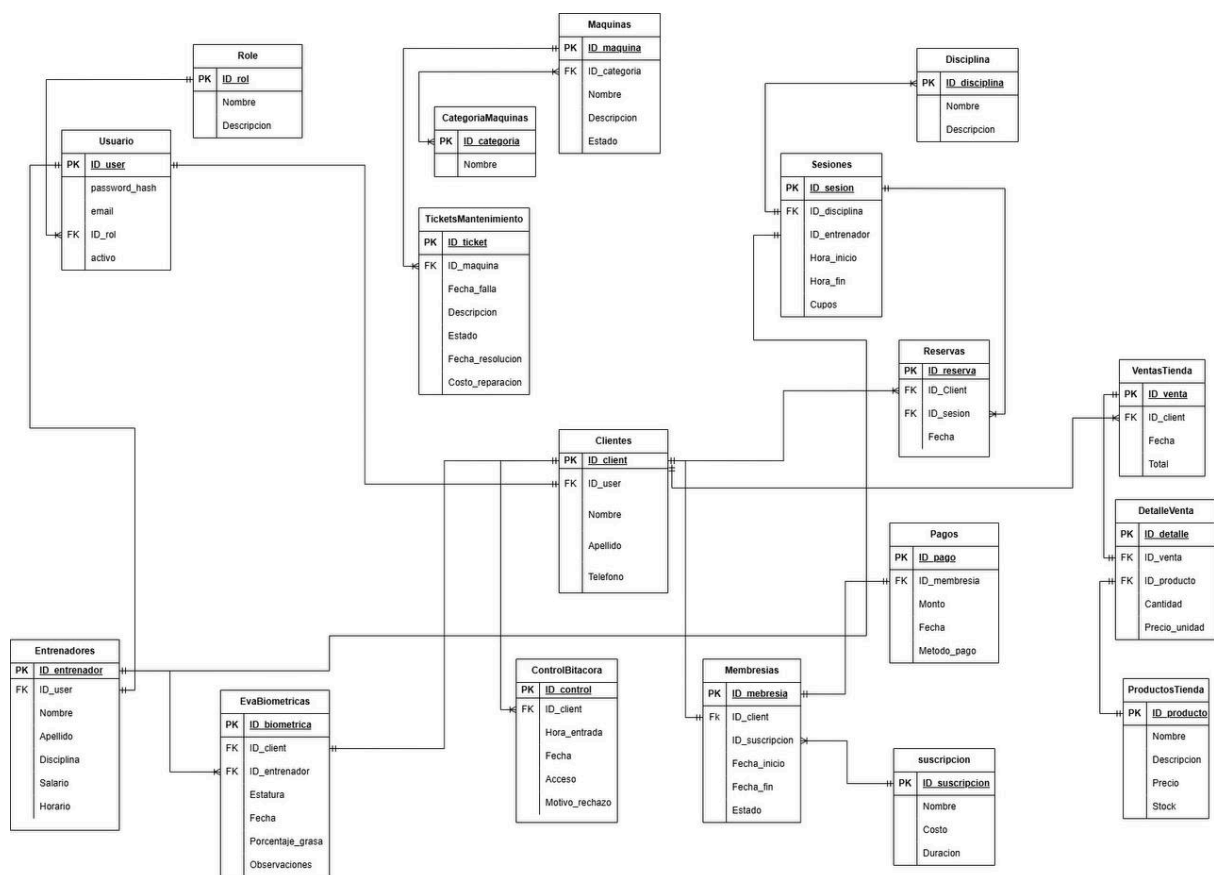


Primera entrega

LABORATORIO

Plataforma API para Gestión Integral de Gimnasios (SmartGym)

1. Diagrama MER completo y normalizado.



2. Alcance del Proyecto

El proyecto **SmartGym** consiste en el diseño, desarrollo e implementación de una **API RESTful** robusta destinada a la gestión integral de centros deportivos. El alcance abarca desde la seguridad de los datos hasta la automatización de procesos operativos críticos.

2.1 Objetivos del Sistema (Inclusiones)

La solución proporcionará servicios para cubrir las siguientes áreas funcionales:

- **Sistema de Identidad y Seguridad:** Implementación de un modelo de autenticación basado en **JWT (JSON Web Tokens)** y autorización por roles (**RBAC**). Esto incluye el registro, inicio de sesión y protección de rutas según los privilegios del usuario (Admin, Finanzas, Entrenador, Cliente).
- **Gestión Académica-Deportiva:** Módulo para la creación de disciplinas, programación de sesiones de entrenamiento y un motor de reservas que valide en tiempo real la disponibilidad de cupos y evite el solapamiento de horarios para los clientes.
- **Gestión Comercial y Financiera:** Control de planes de suscripción, gestión de membresías activas y registro de transacciones de pago para garantizar la trazabilidad económica.
- **Control de Acceso Físico:** Lógica de validación para permitir o denegar la entrada al recinto basada en el estado de cuenta y vigencia del plan del cliente, consultado a través de su número de cédula.
- **Mantenimiento e Inventario de Activos:** Registro del equipamiento del gimnasio, seguimiento de su estado operativo y sistema de tickets para reparaciones y costos asociados.
- **Módulo de Ventas (Retail):** Punto de venta integrado para productos físicos con actualización automática de existencias (stock) en la base de datos PostgreSQL.
- **Seguimiento Biométrico:** Herramientas para que los entrenadores registren y consulten la evolución física de los clientes (peso, estatura, IMC, etc.).

2.2 Límites y Exclusiones (Fuera de Alcance)

Para efectos del lapso académico 2026-1, se definen las siguientes exclusiones:

- **Desarrollo de Interfaz de Usuario (Frontend):** El proyecto se limita estrictamente a la construcción de la capa de servicios (Backend). No incluye aplicaciones web, móviles o interfaces de escritorio.
- **Integración con Hardware Físico:** El control de acceso se simulará mediante endpoints de la API; no se realizará la integración con torniquetes, sensores biométricos o lectores de tarjetas reales.
- **Pasarelas de Pago Reales:** Los pagos se registrarán de forma administrativa en el sistema; no se contempla la conexión con APIs bancarias o procesadores externos (Stripe, PayPal, etc.).
- **Notificaciones Externas:** No se incluye el envío de correos electrónicos o SMS automatizados.

2.3 Entregables Técnicos

- Código fuente documentado en un repositorio de Git.
- Base de datos relacional normalizada en **PostgreSQL**.
- Documentación técnica interactiva mediante **Swagger**.
- Entorno de ejecución estandarizado mediante contenedores **Docker**.

3. Análisis Técnico de Requerimientos

El presente análisis traduce las reglas de negocio del gimnasio **SmartGym** en una estructura técnica lógica, identificando los puntos críticos donde la integridad de los datos y la seguridad son prioridad.

3.1 Descomposición de la Lógica de Negocio (Funcional)

Modelo de Identidad Centralizado (Seguridad):

- **Análisis:** No basta con tener usuarios; el sistema requiere una segregación de funciones. Se optó por una estructura de **Especialización** en el MER (Usuario > Cliente/Entrenador). Esto permite que la autenticación sea única

mediante **JWT**, pero que el sistema pueda cargar perfiles distintos según el ID_rol sin tener datos nulos en la tabla principal.

- **Impacto en el MER:** La relación 1:1 entre Usuarios y Clientes/Entrenadores asegura que un login de cliente no pueda acceder a funciones de administración, cumpliendo con el requerimiento de seguridad.

Validación de Concurrencia y Disponibilidad (Reservas):

- **Análisis:** El desafío no es solo guardar una reserva, sino validar la **Regla de Negocio 2** (Cupos) y la **Regla de Negocio de Solapamiento**. El análisis determinó que el backend debe realizar una consulta transaccional a PostgreSQL antes de insertar en la tabla Reservas.
- **Impacto en el MER:** La tabla Sesiones incluye Cupos y horarios. La tabla Reservas actúa como el contador en tiempo real. La lógica de "solapamiento" se resuelve comparando los intervalos de tiempo (fecha_inicio y fecha_fin) de la nueva reserva contra las reservas existentes del cliente.

Lógica Binaria de Acceso (Torniquetes):

- **Análisis:** El acceso físico es una operación de "solo lectura" de estado. El sistema debe cruzar tres tablas: Cliente > Membresía > Pagos.
- **Impacto en el MER:** Se definió la tabla Control_Acceso no solo para abrir la puerta, sino para generar analítica de rechazos (motivo_rechazo). Esto permite al departamento de finanzas identificar morosidad sin necesidad de revisar recibos físicos.

Gestión de Ciclo de Vida de Activos (Inventario):

- **Análisis:** Las máquinas no son solo registros estáticos; tienen estados. El requerimiento exige que un mantenimiento bloquee el uso.
- **Impacto en el MER:** La tabla Maquina tiene un atributo Estado. El análisis técnico define que la creación de un TicketMantenimiento debe disparar un cambio de estado en la tabla Maquina, asegurando que el inventario refleje la realidad operativa.

3.2 Atributos de Calidad y Restricciones (No Funcionales)

1. Persistencia y Relacionalidad (PostgreSQL):

- **Análisis:** Dado que el sistema maneja transacciones financieras y reservas críticas, el motor relacional es indispensable para garantizar las propiedades **ACID** (Atomicidad, Consistencia, Aislamiento y Durabilidad). El uso de llaves foráneas (FK) impedirá, por ejemplo, borrar a un entrenador que tiene clases programadas.

Seguridad y Statelessness (JWT):

- **Análisis:** Al usar Express y JWT, la API será "stateless" (sin estado). Esto significa que el servidor no guarda sesiones en memoria, lo cual es ideal para el despliegue en **Docker**. El análisis determinó que el token debe contener el ID_rol para evitar consultas innecesarias a la base de datos en cada petición protegida.

Documentación y Contrato de Interfaz (Swagger):

- **Análisis:** Para que el frontend (o los torniquetes físicos) se integren fácilmente, Swagger no es solo ayuda visual; actúa como el "contrato" que define qué datos envía el POS de la tienda o el sistema de biometría, reduciendo errores de integración.

Arquitectura de Respuesta:

- **Análisis:** El requerimiento de códigos HTTP específicos (409 Conflict) obliga a que el análisis de errores en el backend sea preventivo. Se diseñó una estructura de respuesta estandarizada en JSON para que cualquier cliente de la API pueda interpretar si un fallo fue por falta de pago, falta de cupo o error de servidor.

4. Diseño de la API: Listado de Endpoints

4.1 Autenticación y Seguridad

POST /api/v1/auth/login

- Autenticación de credenciales y generación de **Bearer Token (JWT)**.
- **Rol:** Público.

4.2 Catálogos Base del Gimnasio

GET /api/v1/maquinas

- Obtener el listado de máquinas registradas.
- **Rol:** Admin.

GET /api/v1/disciplinas

- Obtener el listado de disciplinas disponibles.
- **Rol:** Admin.

4.3 Sesiones y Reservas

POST /api/v1/sesiones

- Programar una sesión de clase en un horario específico.
- **Rol:** Admin.

GET /api/v1/sesiones

- Obtener el listado de sesiones programadas.
- **Rol:** Administración / Entrenador / Cliente.

GET /api/v1/sesiones?fecha={YYYY-MM-DD}&disciplina={nombre}

- Lista sesiones aplicando filtros.
- **Rol:** Admin/Entrenador/Cliente

POST /api/v1/reservas

- Reservar una clase en un horario específico.
- **Rol:** Cliente.

GET /api/v1/sesiones/{id_sesion}/reservas

- Obtener el listado de asistentes registrados en una sesión.
- **Rol:** Administracion

POST /api/v1/sesiones/{id_sesion}/reservas

- Registrar manualmente una reserva para una sesión específica.
- **Rol:** Admin.

4.4 Control de Acceso Físico

POST /api/v1/accesos/entrada

- Registrar y validar el ingreso del cliente al gimnasio.
- **Rol:** Admin.

4.5 Suscripciones, Membresías y Pagos

GET /api/v1/suscripciones

- Obtener los planes de suscripción disponibles.
- **Rol:** Público.

POST /api/v1/pagos

- Registrar el pago de un cliente.
- **Rol:** Admin.

GET /api/v1/pagos/{id_pago}

- Consultar un pago en específico
- **Rol:** Admin.

GET /api/v1/clientes/{id_cliente}/membresias

- Consultar si la membresía de un cliente está activa o inactiva.

- **Rol:** Administración / Entrenador / Cliente.

4.6 Evaluaciones Físicas y Seguimiento Biométrico

POST /api/v1/clientes/{id_cliente}/evaluaciones

- Registrar una evaluación física del cliente.
- **Rol:** Entrenador.

GET /api/v1/clientes/{id_cliente}/evaluaciones

- Obtener el historial de evaluaciones físicas del cliente.
- **Rol:** Administración / Entrenador / Cliente.

4.7 Tienda, Productos y Ventas

POST /api/v1/productos

- Ingresar mercancía nueva a la tienda.
- **Rol:** Administración.

GET /api/v1/productos

- Obtener el listado de productos disponibles en la tienda.
- **Rol:** Administración.

PATCH /api/v1/productos/{id_producto}/stock

- Modificar el stock de un producto específico.
- **Rol:** Administración.

POST /api/v1/ventas

- Registrar una venta de productos.
- **Rol:** Administración.

GET /api/v1/ventas

- Obtener el historial de ventas registradas.
- **Rol:** Administración.

4.8 Mantenimiento e Inventario de Activos

POST /api/v1/maquinas/{id_maquina}/tickets

- Abrir un ticket de mantenimiento para una máquina específica.
- **Rol:** Administración.

PATCH /api/v1/maquinas/{id_maquina}/estado

- Cambiar el estado operativo de una máquina.
- **Rol:** Administración.

GET /api/v1/tickets

- Obtener el historial general de tickets de mantenimiento.
- **Rol:** Administración.

GET /api/v1/maquinas/{id_maquina}/tickets

- Obtener el historial de reparaciones de una máquina específica.
- **Rol:** Administración.

5. Justificación del Stack Tecnológico

Para el desarrollo de la API de SmartGym, se ha seleccionado un conjunto de tecnologías orientado a garantizar la escalabilidad, la integridad de los datos y el cumplimiento estricto de las reglas de negocio solicitadas.

Entorno de Ejecución y Framework Backend: Node.js y Express.js

- Se optó por Node.js con el framework Express.js dentro de las opciones permitidas.
- **Justificación Técnica:** La arquitectura orientada a eventos y el modelo de entrada/salida sin bloqueo (non-blocking I/O) de Node.js son ideales para manejar múltiples peticiones concurrentes. Esto es críticamente necesario para el módulo de reservas, donde decenas de clientes podrían intentar inscribirse en una clase simultáneamente. Express.js proporciona un enrutamiento ligero y flexible que facilita la implementación de una

arquitectura RESTful limpia y la integración de *middlewares* para la validación de tokens JWT y roles (RBAC).

Motor de Base de Datos: PostgreSQL

- Se seleccionó PostgreSQL para cumplir con la regla de uso de un motor estrictamente relacional.
- **Justificación Técnica:** El dominio de SmartGym requiere un manejo robusto de transacciones financieras (pagos y membresías) y lógicas de exclusión mutua (solapamiento de clases y límite de cupos). PostgreSQL garantiza el cumplimiento de las propiedades ACID (Atomicidad, Consistencia, Aislamiento y Durabilidad). Además, su potente manejo de concurrencia (MVCC) evitará problemas de sobreventa de cupos, y su soporte estricto para llaves foráneas e índices asegura la integridad referencial del inventario y el historial biométrico.

Documentación de la API: Swagger / OpenAPI 3.0

- **Justificación Técnica:** Para cumplir con el estándar exigido, Swagger permite autogenerar una interfaz interactiva y estandarizada. Esto no solo documenta las rutas, los esquemas de entrada (payloads) y las estructuras estrictas de error JSON (ej. 400 y 409), sino que actúa como un contrato claro para futuras integraciones con aplicaciones frontend o dispositivos de control de acceso físico (torniquetes).

Entorno de Despliegue: Docker y Docker Compose

- **Justificación Técnica:** Cumpliendo con el requerimiento de contenerización, Docker aísla la aplicación y sus dependencias de la máquina anfitriona. El uso de docker-compose permite orquestar simultáneamente el contenedor de la API en Node.js y el contenedor del motor PostgreSQL. Esto garantiza la paridad entre entornos: el proyecto se ejecutará de forma idéntica en las máquinas de desarrollo y en el equipo del evaluador durante la revisión.

6. Validaciones requeridas para cumplir con las reglas de negocio

Esta sección define las validaciones que debe aplicar la API para garantizar el cumplimiento de las reglas de negocio principales del sistema **SmartGym**.

Para los errores de conflicto, la API debe responder con el código HTTP **409 Conflict** y la siguiente estructura JSON:

```
{
  "error": "Conflict",
  "codigoInterno": "CODIGO_DE_LA_REGLA",
  "mensaje": "Descripción del fallo",
  "timestamp": "ISO-8601-FORMAT"
}
```

6.1 Validación de solapamiento de reservas

- **Endpoint:** `POST /api/v1/reservas`
- **Regla:** El sistema debe bloquear la inscripción si el cliente ya posee otra clase reservada en el mismo bloque horario.
- **Código interno:** `ERR_RESERVA_SOLAPAMIENTO`

Ejemplo de respuesta:

```
{
  "error": "Conflict",
  "codigoInterno": "ERR_RESERVA_SOLAPAMIENTO",
  "mensaje": "Ya posee una clase reservada a la misma hora",
  "timestamp": "2026-04-24T12:00:00Z"
}
```

6.2 Validación de cupos disponibles

- **Endpoint:** `POST /api/v1/reservas`
- **Regla:** El sistema debe rechazar la solicitud si el número de reservas actuales alcanza el límite máximo de cupos de la sesión.

- **Código interno:** `ERR_RESERVA_CAPACIDAD_MAXIMA`

Ejemplo de respuesta:

```
{
  "error": "Conflict",
  "codigoInterno": "ERR_RESERVA_CAPACIDAD_MAXIMA",
  "mensaje": "No se encuentran cupos disponibles para esta sesión",
  "timestamp": "2026-04-24T12:00:00Z"
}
```

6.3 Validación de acceso físico

- **Endpoint:** `POST /api/v1/accesos/entrada`
- **Regla:** El sistema debe denegar el ingreso si el cliente no posee una membresía pagada y vigente.
- **Código interno:** `ERR_ACCESO_MEMBRESIA_VENCIDA`

Ejemplo de respuesta:

```
{
  "error": "Conflict",
  "codigoInterno": "ERR_ACCESO_MEMBRESIA_VENCIDA",
  "mensaje": "Estimado usuario, usted se encuentra con un estado de membresía vencida",
  "timestamp": "2026-04-24T12:00:00Z"
}
```

6.4 Validación de stock

- **Endpoint:** `POST /api/v1/ventas`
- **Regla:** El sistema debe impedir la venta si la cantidad solicitada supera las existencias disponibles en inventario.
- **Código interno:** `ERR_STOCK_INSUFICIENTE`

Ejemplo de respuesta:

```
{
  "error": "Conflict",
  "codigoInterno": "ERR_STOCK_INSUFICIENTE",
  "mensaje": "La cantidad solicitada a vender no se encuentra disponible",
  "timestamp": "2026-04-24T12:00:00Z"
}
```

6.5 Validación de inmutabilidad de pagos

- **Endpoint:** `POST /api/v1/pagos`
- **Regla:** Una vez procesado un pago, el sistema debe restringir cualquier modificación (`PUT/PATCH`) o eliminación (`DELETE`) del registro financiero para garantizar la auditoría.
- **Código interno:** `ERR_PAGO_INMUTABLE`

Ejemplo de respuesta:

```
{
  "error": "Conflict",
  "codigoInterno": "ERR_PAGO_INMUTABLE",
  "mensaje": "Se encuentra prohibido modificar o eliminar cualquier pago",
  "timestamp": "2026-04-24T12:00:00Z"
}
```

6.6 Validación de estado de activos

- **Endpoint:** `POST /api/v1/maquinas/{id_maquina}/tickets`
- **Regla:** Al abrir un ticket de mantenimiento, el sistema debe cambiar automáticamente el estado de la máquina a `En Mantenimiento` , bloqueando su disponibilidad operativa.
- **Código interno:** `ERR_MAQUINA_MANTENIMIENTO`

Ejemplo de respuesta:

```
{  
  "error": "Conflict",  
  "codigoInterno": "ERR_MAQUINA_MANTENIMIENTO",  
  "mensaje": "La máquina se encuentra en mantenimiento",  
  "timestamp": "2026-04-24T12:00:00Z"  
}
```